

UNIVERSITY FOR THE CREATIVE ARTS

**Portfolio**

FGCT4025 Programming & GUI APPLICATIONS  
Mountain Correct Game

**Bishant Gharti Chhetri**

BSc Computer Science / Creative Computing

July2025

I, Bishant Gharti Chhetri, confirm that the work presented in this portfolio is my own. Where information has been derived from other sources, I confirm that this has been indicated in the portfolio.

© 2025, Bishant Gharti Chhetri  
School of Games & Creative Technology  
University for the Creative Arts

## Contents

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>Mountain Correct Game .....</b>  | <b>4</b>  |
| 1.1      | Introduction.....                   | 4         |
| <b>2</b> | <b>User Manual.....</b>             | <b>5</b>  |
| 2.1      | Running in NetBeans .....           | 5         |
| 2.2      | Playing Mountain Correct Game ..... | 6         |
| <b>3</b> | <b>Code Description.....</b>        | <b>8</b>  |
| 3.1      | Graphical User Interface .....      | 8         |
| 3.2      | Event Handling .....                | 8         |
| 3.3      | Inheritance.....                    | 8         |
| 3.4      | User-defined Class .....            | 9         |
| 3.5      | Method.....                         | 9         |
| 3.6      | Repetition.....                     | 10        |
| 3.7      | Selection.....                      | 10        |
| <b>4</b> | <b>Reflection .....</b>             | <b>11</b> |
|          | <b>Bibliography .....</b>           | <b>12</b> |
|          | <b>UML Diagram .....</b>            | <b>13</b> |

# 1 Mountain Correct Game

## 1.1 Introduction

The Mountain Correct Game is a memory matching card game where players must find different types of matching mountain images. This project was developed as part of the Programming & GUI Applications module to demonstrate proficiency in Java Swing, event handling, and object-oriented programming concepts. And the game features a 4x3 grid of cards (12 total, 6 matching pairs). It also has Score tracking based on completion time and number of attempts. It shows Visual feedback for matches and mismatches. It has Timer to track game duration & reset button to start a new game. I choose this project because it combines fundamental programming concepts with an engaging user interface, while allowing for creative expression through the mountain theme.

It will helps me to Implement robust input to Create clear user feedback mechanisms for Develop a complete project from start to finish with The skills gained from this project will serve as a strong foundation for when I revisit my upcoming project concept in the future with more advanced Java knowledge.

## 2 User Manual

### 2.1 Running in NetBeans

When we Launch NetBeans then Open the NetBeans IDE on your computer and Import the Project by Click File & Open Project in the menu bar then Navigate to the folder containing the unzipped project files. We Select the project folder (should show Java project icon) then Click Open. We must Verify Project Structure In the Projects pane, expand Source Packages, Mountain Correct Game. We Should contain MountainCorrectGame.java. It helps Libraries should show Java JDK:

1. Download and unzip the project folder containing all source files and images
2. Open NetBeans IDE
3. Select File > Open Project and navigate to the project folder
4. Ensure all mountain image files are in the correct package directory (/mountaincorrectgame/)
5. Right-click on the project in the Projects pane and select "Run"
6. The game window will appear, ready to play

## 2.2 Playing Mountain Correct Game



Figure 1: When the game launches, you'll see the main interface

### Gameplay Steps:

1. Click any card to reveal a mountain image
2. Click a second card to try to find its match
3. If the images match:
  - Both cards remain visible (turned green)
  - Your score increases
4. If the images don't match:
  - Both cards flip back after 1 second
  - Your attempts counter increases
5. Continue until all pairs are found
6. The game ends automatically, showing your final score and time

**Controls:**

- **Restart Game:** Resets the game board, score, and timer
- **Check Guess:** Displays detailed game instructions

## 3 Code Description

### 3.1 Graphical User Interface

The GUI uses Swing components extensively:

- JButton for all card elements and control buttons
- JLabel for displaying score, timer, attempts, and status messages
- ImageIcon to handle mountain images
- Color constants for visual feedback (blue for unflipped cards, green for matches)

```
// Example of GUI component initialization
private void resetCardAppearance(JButton button) {
    button.setIcon(null);
    button.setText("?");
    button.setBackground(CARD_BACK_COLOR); // Blue color
    button.setPreferredSize(new Dimension(CARD_WIDTH, CARD_HEIGHT));
}
```

### 3.2 Event Handling

The game uses ActionListener for all button interactions which helps Card buttons use a shared CardButtonListener class where Control buttons have dedicated listeners and javax.swing.Timer handles delayed card flipping and game timing.

```
// Card selection handling
private class CardButtonListener implements ActionListener {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (!gameInProgress) return;

        JButton clickedButton = (JButton) e.getSource();
        if (!clickedButton.isEnabled() || clickedButton == firstSelection) return;

        flipCard(clickedButton);

        if (firstSelection == null) {
```

```

        firstSelection = clickedButton;
    } else if (secondSelection == null) {
        secondSelection = clickedButton;
        checkMatch();
    }
}
}

```

### 3.3 Inheritance

The main game class MountainCorrect extends javax.swing.JFrame, inheriting all window management functionality from Swing's frame class. This provides the basic window structure without needing to implement it from scratch.

### 3.4 User-defined Class

The game consists of one main class (Mountain Correct) that handles all game logic and UI. Key components include Card management (flipping, matching) and Score calculation. And which also show timer functionality. It helps to track Game state. The game has all functionality which contained within this single class as the game's complexity didn't warrant multiple classes. The class uses several helper methods to organize functionality logically. It helps game to play smooth.

### 3.5 Method

The Key methods that include in game:

- setupGame (): Initializes all game components
- resetGame (): Prepares a new game
- flipCard()/flipCardBack():Handle card state changes
- checkmatch (): Compares selected cards
- handleMatch ( )/ handleMismatch( ) : Process match results
- Various update methods for UI elements

```

private void handleMatch() {
    pairsFound++;
    score += Math.max(0, BASE_SCORE - (attempts /
SCORE_PENALTY_PER_ATTEMPT));
    updateScoreDisplay();

    firstSelection.setEnabled(false);
    secondSelection.setEnabled(false);
    firstSelection.setBackground(MATCHED_CARD_COLOR); // Green color

    if (pairsFound == TOTAL_PAIRS) {
        endGame();
    }
}

```

### 3.5 Repetition

The game helps me to uses loops to initialize and reset card buttons While loop conceptually in the game timer (implemented via Timer class) Collection with enhanced for loops

*// Example of card initialization loop*

```

for (JButton button : cardButtons) {

    button.addActionListener(new CardButtonListener());

    resetCardAppearance(button);

}

```

### 3.6 Selection

The game primarily uses if-else statements where it Determining game state and also Handling card selections. It helps to Manage match outcomes and Control game flow.

```
if (firstIndex != null && secondIndex != null &&
firstIndex.equals(secondIndex)) {
    handleMatch();
} else {
    handleMismatch();
}
```

*Code 1: Using null*

## 4 Reflection

This project significantly improved my understanding of Java Swing and GUI development were Event driven programming Game state management which helps Code organization and structure

The Challenges included Properly timing card flips (solved with Timer) Managing game state during player interactions which Ensuring image resources loaded correctly. The project shows the importance of Clear code organization and Proper event handling. It helps User feedback in interactive applications

The Future improvements could include in Difficulty levels High score tracking which Animated card flips and Sound effects.

## Bibliography

- Oracle Java Documentation
- Swing Tutorials from JavaTpoint
- "Head First Java" by Kathy Sierra and Bert Bates

## UML Diagram

